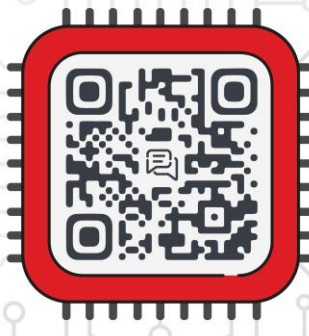




5



13



1560 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

AVL trees

د. عملي مشترك



RB Informatics; 7/5/2025

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

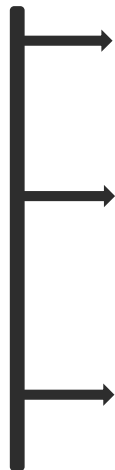
تحدثنا في المحاضرة السابقة عن مشكلة تتعرض لها في شجرة البحث الثنائي وهذه المشكلة هي التوازن، إذ سيختلف التعقيد الزمني للعمليات الأساسية في شجرة البحث الثنائي وفقاً لتوازن الشجرة وستنحصر بطبيعة الحال امتيازاتها، وموضوع محاضرة اليوم يتحدث عن أحد حلول هذه المشكلة وهو أحد أول وأقدم الحلول لها.

لنتصور أن لدينا شجرة بحث ثنائية ما، وفقاً لتعريف التوازن الذي تحدثنا عنه في المحاضرة السابقة تكون الشجرة الثنائية متوازنة إذا كان:

الفرق بين ارتفاع ابنها الأيمن وابنها الأيسر قيمة صغيرة (إما صفراً أو واحداً).

حيث: الارتفاع هو أطول مسار من الجذر الذي نقف عنده حتى الأوراق، وبالتالي يمكننا من خلال مفهوم الارتفاع تحديد ما إذا كانت شجرة ما متوازنة أم لا، يعني "لو كان لدينا شجرة فيها ارتفاع معلوم لكل عقدة يمكننا تحديد هل الـ TREE متوازنة أم لا" ويمكن كذلك معرفة هذا الأمر لكل SUBTREE ضمن هذه الشجرة الكبيرة، وهذا هو المبدأ الذي سننطلق منه في محاولة موازنة شجرة البحث الثنائية من الآن فصاعداً سنخزن في كل عقدة attribute إضافية تحوي الارتفاع عند العقدة التي نقف عندها.

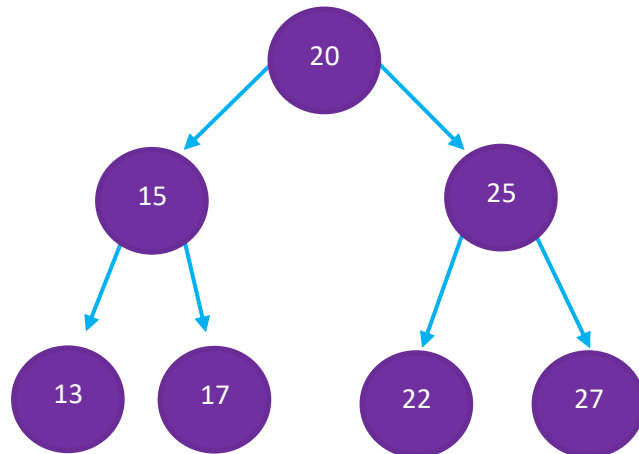
Height



Height : 2

Height : 1

Height : 0



تعريف :

avl tree : هي شجرة بحث ثنائية تلقائية التوازن تحقق أنه: عند كل عقدة في الشجرة فإن القيمة المطلقة للفرق بين ارتفاع الابن الأيمن و الابن الأيسر لا تتجاوز الـ (1) ← محصورة بالمجال $[-1,1]$ ، و نسمي هذا الفرق الذي نختبره بـ balance factor.

- الشجرة المرسومة بالأعلى هي avl tree لأنها في كل عقدة سنجد قيمة الـ balance factor تساوي الـ 0 محققةً للشرط الخاص بالشجرة.

الآن تبقى الجزء الأهم من التعريف (تلقائية التوازن)، ما هي الوسيلة التي ستلجأ

لها الشجرة حتى تقوم بعملية التوازن التلقائي؟

الآن بعد إضافة خاصية الارتفاع لكل عقدة في الشجرة يمكننا التحقق من اختلال توازنها دوماً في أي نقطة من النقاط، من خلال اختبار ارتفاع الابن الأيمن والابن الأيسر و الحظي بالفرق بينهما، بعد ذلك أي بعد اكتشاف الخلل سيتم إصلاحه من خلال مفهوم جديد يسمى الـ rotations أو التدوير.

ما معنى rotation؟

عملية نعيد من خلالها ترتيب العقد بصورة لا تخرق شروط شجرة البحث الثنائي (العملية سميت بهذا الاسم لأنها تشابه فعلياً الدوران حول نقطة بجهة معينة).

ولهذا الدوران عدة أشكال: left rotation – right rotation – left-right rotation – right-left rotation

ملاحظة قبل البدء بالشرح:

المثلثات تشير لـ SUBTREES والدوائر تشير لعقد

1. Left rotation

ليكن لدينا الشجرة التالية إذا أردنا القيام ب LEFT ROTATION على العقدة A، فيمكن تصور الأمر على أنه "نثبت الابن الأيمن لها B" كمرکز للدوران وندور العقد باتجاه اليسار، **أي:** ستدور A حول B باتجاه اليسار (وهذا يعني أن A ستصبح ابن B الأيسر).

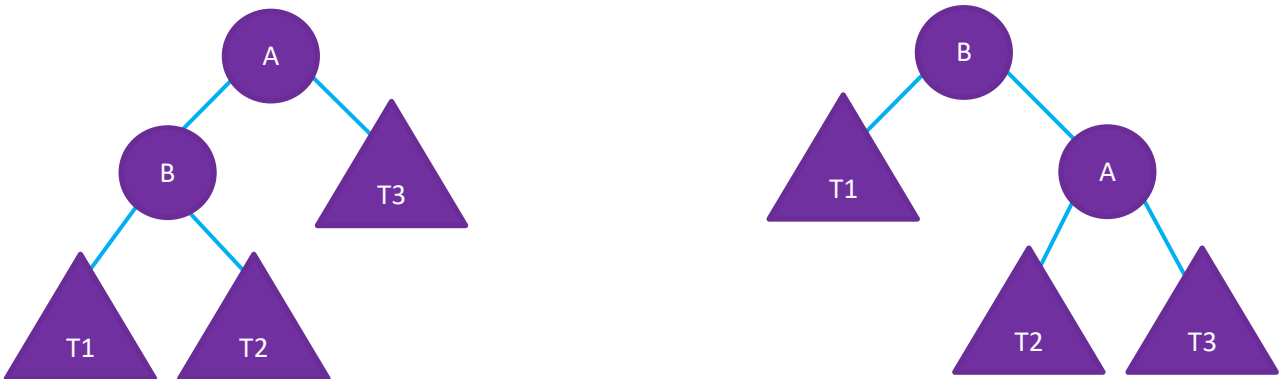
- حسناً لكن ماذا سيحدث ل T2 (الابن الأيسر الحالي ل B)؟ سيصبح الابن الأيمن ل A
- لما ذلك؟ لأن T2 جميع القيم التي تحويها أكبر من A وأصغر من B وهذا وفقاً لخصائص ال BST والآن لنشاهد كيف ستتم العملية.



2. right rotation

■ ليكن لدينا الشجرة التالية إذا أردنا القيام ب RIGHT ROTATION على العقدة A، فيمكن تصور الأمر على أنه "نثبت الابن الأيسر لها B" كمرکز للدوران وندور ال A باتجاه اليمين، **أي:** ستدور A حول B باتجاه اليمين (وهذا يعني أن A ستصبح ابن B الأيمن).

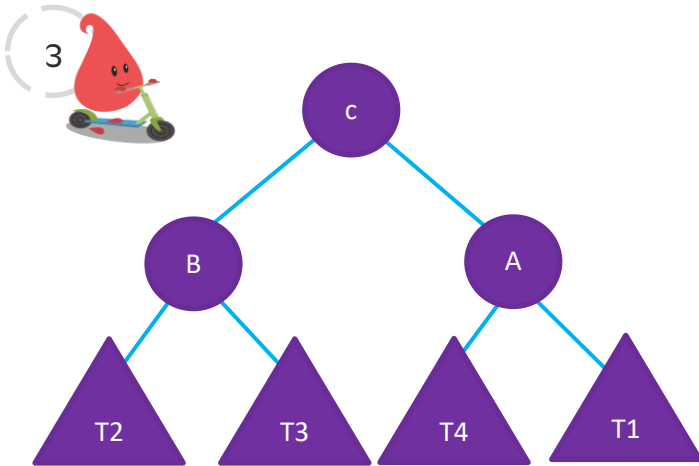
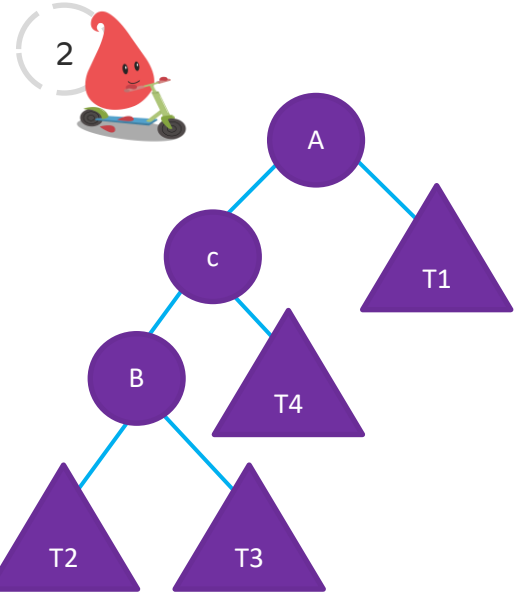
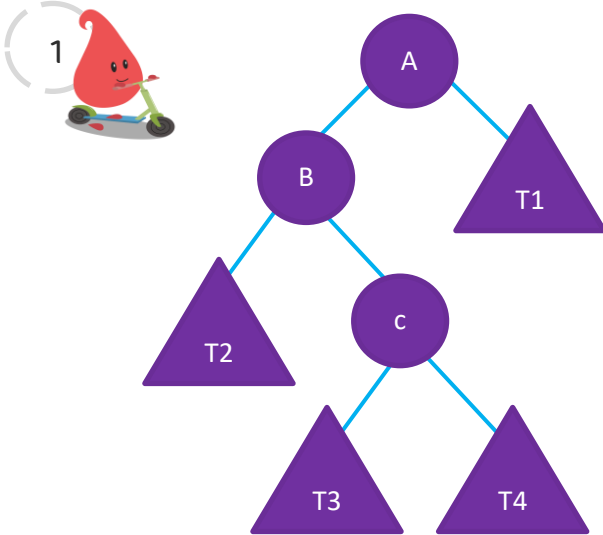
- حسناً لكن ماذا سيحدث ل T2 (الابن الأيمن الحالي ل B)؟ سيصبح الابن الأيسر ل A.
- لما ذلك؟ لأن T2 جميع القيم التي تحويها أصغر من A وأكبر من B وهذا وفقاً لخصائص ال BST والآن لنشاهد كيف ستتم العملية.

وكما نلاحظ أن

العملية الأولى هي عكس الثانية تماماً، والنوعان المتبقيان لل ROTATIONS يعتمدان تماماً دمج هذين النوعين للقيام بهدف معين الذين تحدثنا عنهما (مهم فهم النوعين الي فوق بشكل كامل لفهم النوعين الذين في الأسفل).

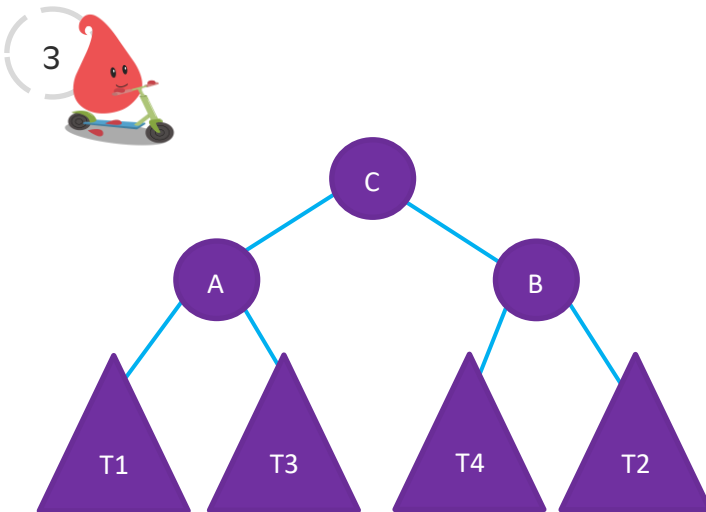
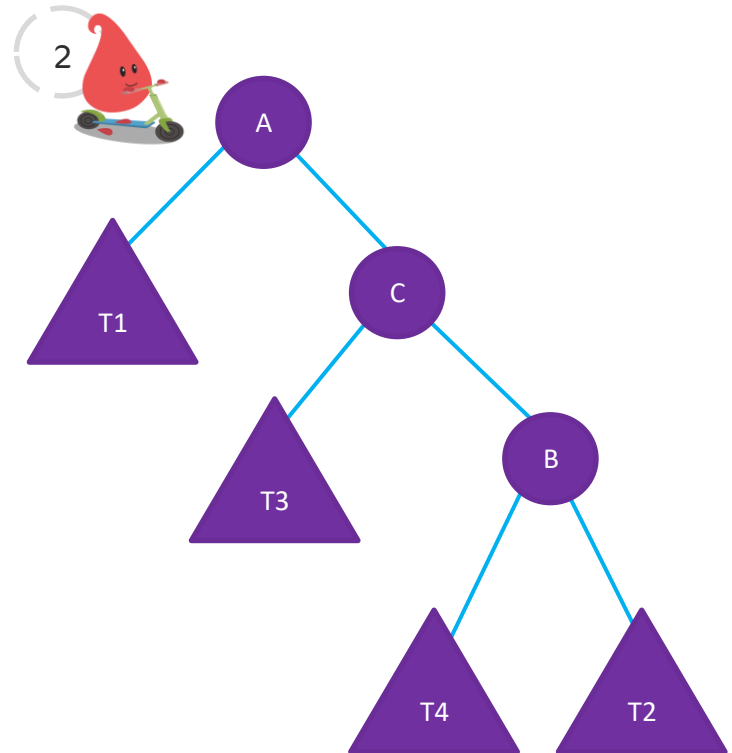
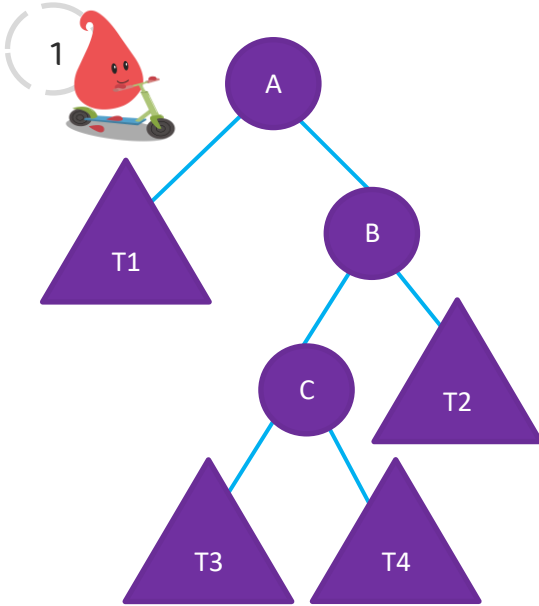
إذا أردنا أن نقوم بـ left-right rotation على A في الشجرة التالية

- بداية الأمر نقوم بـ left rotation على الابن الأيسر للعقدة (B) بحيث ستصبح (C) الابن الأيمن لـ (B) مركز دوران ثم ندور B باتجاه اليسار (أصبحت B هي الابن الأيسر لـ C).
- ثم نقوم بـ right rotation على العقدة ذاتها بحيث ستصبح C (الابن الأيسر الجديد لـ A) محور دوران وسندور A حولها باتجاه اليمين (ستصبح العقدة C هي الجذر)



إذا أردنا أن نقوم بـ RIGHT-LEFT rotation على A في الشجرة التالية

- بداية الأمر نقوم بـ RIGHT rotation على الابن الأيمن للعقدة (B) بحيث ستصبح (C) الابن الأيسر لـ (B) مركز دوران ثم ندور B باتجاه اليمين (أصبحت B هي الابن الأيمن لـ C).
- ثم نقوم بـ right rotation على العقدة ذاتها بحيث ستصبح C (الابن الأيمن الجديد لـ A) محور دوران و سندور A حولها باتجاه اليمين (ستصبح العقدة C هي الجذر)





❧ يأتي سؤال هام في ذهن الطالب، متى نقوم بتطبيق هذه الـ ROTATIONS وهل هنالك حالات محددة بحيث يمكننا معرفة متى نستخدم نوعاً ما و متى لا نستخدمه؟

بالطبع يوجد حالات، لقد اتفقنا أن اختلال التوازن يكون إذا كانت القيمة المطلقة للفرق بين ارتفاع الشجرة اليمنى وارتفاع الشجرة اليسرى هو أكبر من 1.

لتكن لدينا عقدة لتكن A ابنها الأيمن B وابنها الأيسر C، في شجرة بحث ثنائي ولديها اختلال في التوازن

فتميز الحالات الآتية:

- إذا كان اختلال التوازن ناجماً عن كون الارتفاع في الشجرة اليسرى التابعة لـ C أكبر من الشجرة اليمنى التابعة لـ B بحيث أصبح الفرق بالقيمة المطلقة أكبر من واحد $|C - B| > 1$ فتميز حالتين فرعتين، ننظر إلى الابن الأيسر C ونفحص ارتفاع أبنائه:

1. لو كان الارتفاع عند الابن الأيسر لـ C أكبر أو يساوي الارتفاع عند ابنها الأيمن: نقوم بـ RIGHT ROTATION على A وتسمى أيضاً LL Case (Left-Left) وباختصار عندما تكون العقدة غير المتزنة في اليسار، والمشكلة قادمة من ابنها اليسار أيضاً.

2. لو كان الارتفاع عند الابن الأيمن لـ C أكبر من الارتفاع عند ابنها الأيسر: نقوم بـ left-right rotation على A . (أي نقوم بـ LEFT ROTATION على C ثم نقوم بـ RIGHT ROTATION على A) . وتسمى أيضاً LR Case (Left-Right) وباختصار عندما تكون العقدة غير المتزنة في اليسار، لكن المشكلة قادمة من ابنها اليمين.

- إذا كان اختلال التوازن ناجماً عن كون الارتفاع في الشجرة اليمنى التابعة لـ B أكبر من الشجرة اليمنى التابعة لـ C بحيث أصبح الفرق بالقيمة المطلقة أكبر من واحد. فتميز حالتين فرعتين:

3. لو كان الارتفاع عند الابن الأيمن لـ B أكبر أو يساوي الارتفاع عند ابنها الأيمن : نقوم بـ left ROTATION على A. وتسمى أيضاً RR Case (Right-Right) وباختصار عندما تكون العقدة غير المتزنة في اليمين، والمشكلة قادمة من ابنها اليمين أيضاً.

4. لو كان الارتفاع عند الابن الأيسر لـ B أكبر من الارتفاع عند ابنها الأيسر : نقوم بـ right-left rotation على A .
(أي نقوم بـ right ROTATION على B ثم نقوم بـ left ROTATION على A) .
- وتسمى ايضاً: RL Case (Right-Left) وباختصار عندما تكون العقدة غير المتزنة في اليمين، لكن المشكلة قادمة من ابنها اليسار.

هذه هي الأربع حالات والتي بشكل عام سنقوم باستخدامها في بناء شجرة الـ AVL.
والجدول التالي يوضح الفرق

الحالة (Case)	المكان غير المتزن	اتجاه المشكلة	نوع الدوران المستخدم
LL	اليسار	اليسار	دوران يميني (Right Rotation)
LR	اليسار	اليمن	يساري ثم يميني (Left → Right)
RR	اليمن	اليمن	دوران يساري (Left Rotation)
RL	اليمن	اليسار	يميني ثم يساري (Right → Left)

سؤال: ما هي العمليات التي يمكن أن تجعل توازن شجرة البحث الثنائي تختل؟

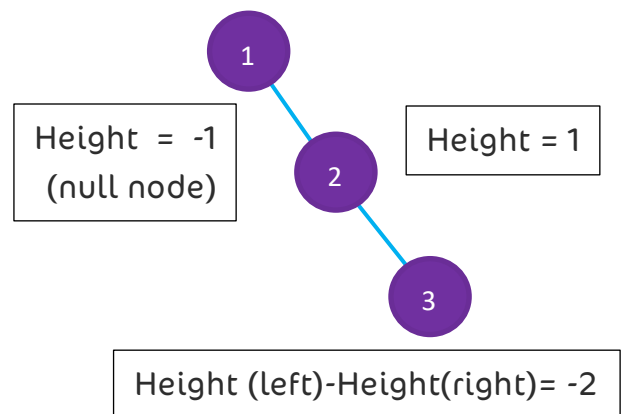
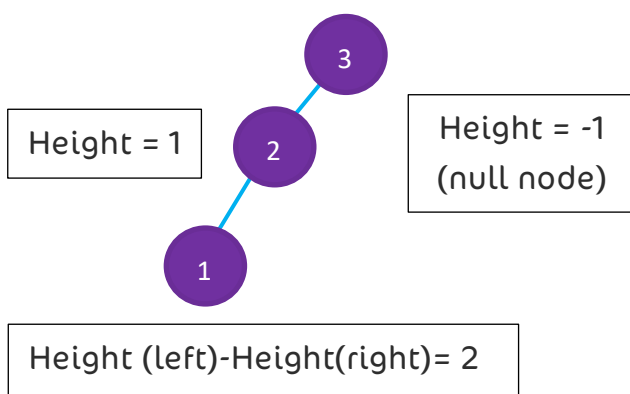
الحذف والإضافة بالتالي عند حذفنا لعقدة ما أو إضافة عقدة ما علينا دوماً التأكد من أن التوازن مصون وإن لم يكن كذلك سوف نقوم بتصحيح الشجرة من خلال الـ rotations بحيث يعود التوازن.

والآن سنستعرض بشكل عام عن كيف يمكن تمييز الحالات؟

حتى نعرف ما إذا كانت الشجرة اليمنى هي سبب المشكلة أو الشجرة اليسرى هي سبب المشكلة سنقوم بأخذ هذا الفرق:

$$\text{height of the left tree} - \text{height of the right tree}$$

فإن كانت نتيجة هذا الفرق أكبر من الـ 1 فهذا يعني أن الشجرة اليسرى هي التي تسببت بالخلل
وإن كانت النتيجة أصغر من الـ -1 فهذا يعني أن الشجرة اليمنى هي التي تسببت بالخلل





في حال حددنا أن التوازن عند عقدة ما مختل، وحددنا أي من الأبناء فيه المشكلة وليكن x بنفس الطريقة سنتأكد: إن كان الارتفاع عند الابن الأيمن $-x$ أكبر أو الارتفاع عند الابن الأيسر $-x$ أكبر

نأخذ الفرق بين height of the left tree - height of the right tree

ونقارن هذه المرة بالنسبة لـ 0 كونه لا يهمنا اختلال التوازن عند هذه العقدة، بل يهمنا معرفة في أي جهة بالنسبة لهذه العقدة يوجد عدد عقد أكبر

لنفترض أن x كان الابن الأيسر للعقدة التي فيها الخلل اليسرى، فنقوم بأخذ الفرق بين

$HEIGHT(ROOT.LEFT.LEFT) - HEIGHT(ROOT.LEFT.RIGHT)$

لو كان أكبر أو يساوي 0 فنقوم بـ RIGHT ROTATION

لو كان أصغر من الـ 0 فنقوم بـ left-right rotation

ملاحظة:

من الممكن الاستفادة من الـ value الذي نضيفه في عملية الإضافة لمعرفة أي الطرفين فيه عدد عقد أكبر (كوننا نعرف أين تمت إضافة هذه القيمة بالنسبة لأي عقدة " لو كانت القيمة المضافة أكبر من قيمة العقدة فهي على يمينها وإن كانت أصغر فيه على يسارها")، وهذا يجعلنا نعرف في أي طرف بالنسبة لكل عقدة حدثت زيادة العدد فيمكننا الاستعاضة عن الطريقة السابقة، وعلى هذا المبدأ سنعتمد في كتابة خوارزميتنا وسيتم التحقق من التوازن أثناء كل استدعاء لعمليتي الحذف أو الإضافة

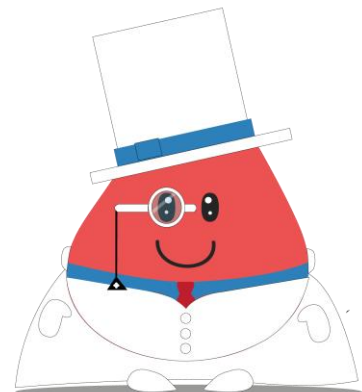
الآن لنبدأ بكتابة بنية المعطيات و خصائصها ثم سنقوم بشرح مثال عملي

1

```

class AVL {
    class NODE {
        char key;
        int height;
        NODE left, right;
        NODE(char key) {
            left=right=null;
            this.key =key;
            height =0;
        }
    }
    int height(NODE node) {
        // returns the height of the actual node and -1 for nulls
        return (node == null) ? -1 : node.height;
    }
    int getBalance(NODE node) {
        // this will return (the height of the left tree - the height of the right tree)
        // if the node is null return 0 since both children are null (height -1)
        return (node ==null) ?0 : height(node.left) - height(node.right);
    }
    NODE rightRotate(NODE y) {
        //left child of the original node
        NODE x = y.left;
        // the right child of the left child of the original node
        NODE T2 = x.right;
        // the right child of the left child of the original node is the original node
        x.right = y;
        // the new left child of the original node (T2)
        y.left = T2;
        // new heights
        y.height = Math.max(height(y.left), height(y.right)) + 1;
        x.height = Math.max(height(x.left), height(x.right)) + 1;
        //this returns the left child (it will become the new root)
        return x;
    }
}

```

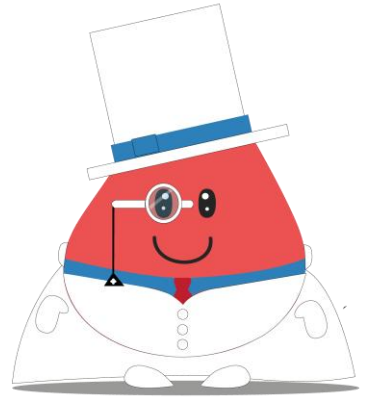


2

```

NODE leftRotate(NODE y) {
    //right child of the original node
    NODE x = y.right;
    // the left child of the right child of the original node
    NODE T2 = x.left;
    // the left child of the right child of the original node is the original node
    x.left = y;
    // the new right child of the original node (T2)
    y.right = T2;
    // new heights
    x.height = Math.max(height(x.left), height(x.right)) + 1;
    y.height = Math.max(height(y.left), height(y.right)) + 1;
    //this returns the right child (it will become the new root)
    return x;
}

```



3

```

NODE insert(NODE node, char key) {
    //common insertion
    if (node == null) return new NODE(key);
    if (key < node.key)
        node.left = insert(node.left, key);
    else if (key > node.key)
        node.right = insert(node.right, key);
    else
        return node;
    //calculate the new height
    node.height = 1 + Math.max(height(node.left),
    height(node.right));
    //the difference between the heights of the
    children
    int balance = getBalance(node);
    //right rotation case
    if (balance > 1 && key < node.left.key)
        return rightRotate(node);

```

4

```

//left rotation case
if (balance < -1 && key > node.right.key)
    return leftRotate(node);
//left-right rotation case
if (balance > 1 && key > node.left.key) {
    node.left = leftRotate(node.left);
    return rightRotate(node);
}
//right-left rotation case
if (balance < -1 && key < node.right.key)
{
    node.right = rightRotate(node.right);
    return leftRotate(node);
}
return node;
}

```

5

```

NODE minValueNode(NODE node) {
    NODE current = node;
    while (current.left != null)
        current = current.left;
    return current;
}

NODE delete(NODE root, char key) {
    // common deletion
    if (root == null)
        return root;
    if (key < root.key)
        root.left = delete(root.left, key);
    else if (key > root.key)
        root.right = delete(root.right, key);
    else {
        if((root.left==null)&&(root.right==null))
            return null;
        else if ((root.left == null))
            return root.right;
        else if ((root.right == null))
            return root.left;
        else {
            NODE temp =
minValueNode(root.right);
            root.key = temp.key;
            root.right = delete(root.right,
temp.key);
        }
    }
}

```

6

```

if (root == null)
    return root;
//calculate the new height
root.height = 1 + Math.max(height(root.left),
height(root.right));
//the difference between the heights of
the children
int balance = getBalance(root);
//right rotation case
if (balance>1&&getBalance(root.left)>= 0) {
    return rightRotate(root);
}
//left-right rotation case
if (balance>1&& getBalance(root.left) < 0) {
    root.left = leftRotate(root.left);
    return rightRotate(root);
}
//left rotation case
if(balance<-1&&getBalance(root.right)<=0) {
    return leftRotate(root);
}
//right-left rotation case
if(balance<-1&&getBalance(root.right) > 0) {
    root.right = rightRotate(root.right);
    return leftRotate(root);
}
return root;

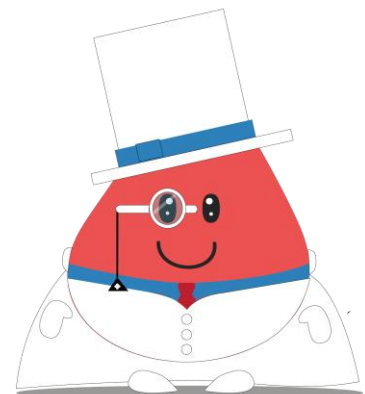
```

7

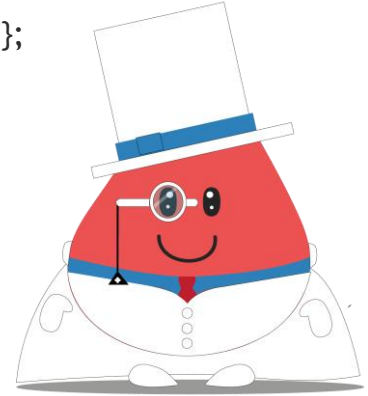
```

void inorder(NODE root) {
    if (root != null) {
        inorder(root.left);
        System.out.print(root.key+" ");
        inorder(root.right);
    }
}

```



```
public class Main{
    public static void main(String[] args) {
        AVL tree = new AVL();
        AVL.NODE root = null;
        char[] values = {'H','I','J','B','A','E','C','F','D','G','K','L'};
        for (char val : values) {
            root = tree.insert(root, val);
            tree.inorder(root);
            System.out.println();
        }
    }
}
```

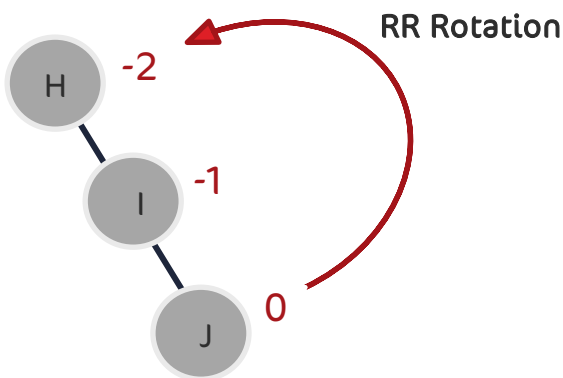


مثال عملي

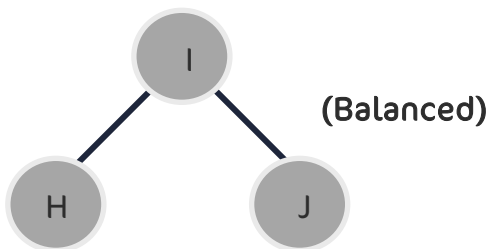
1. Construct an AVL tree having the following elements

H, I, J, B, A, E, C, F, D, G, K, L

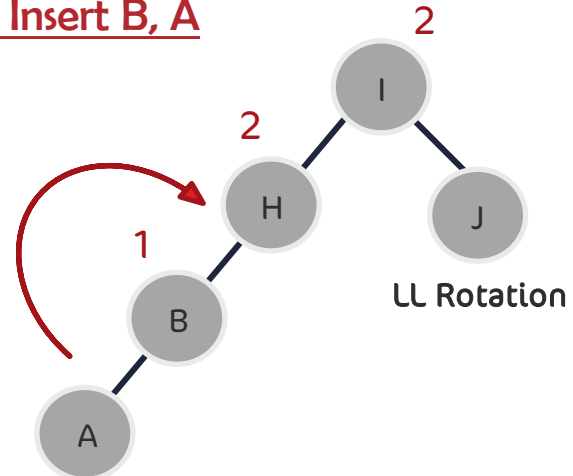
1. Insert H, I, J



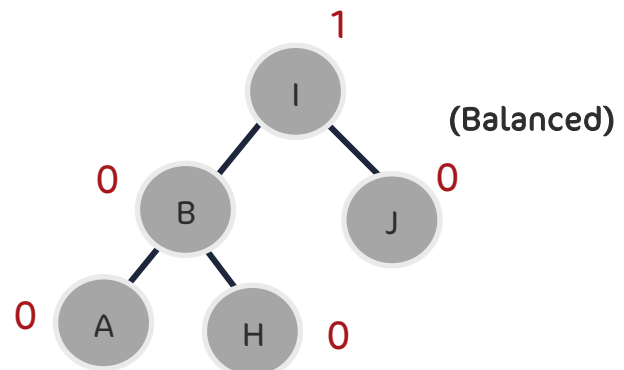
The resultant balance tree is:



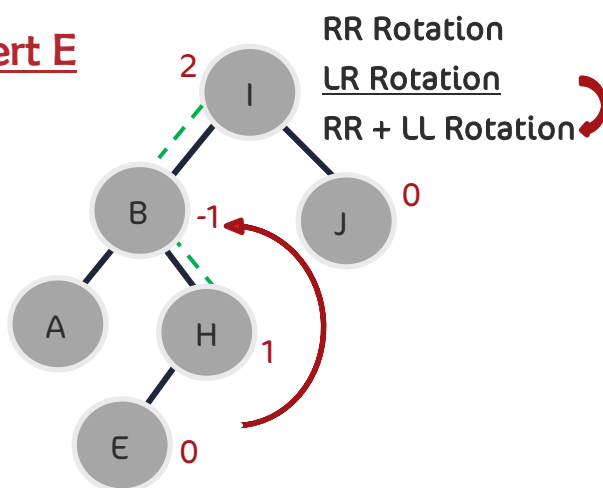
2. Insert B, A



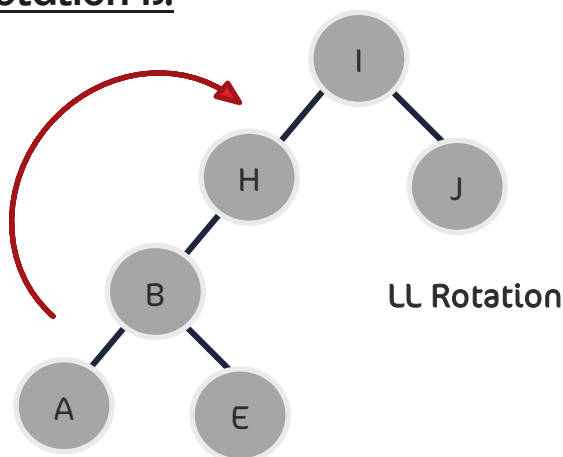
The resultant balance tree is:



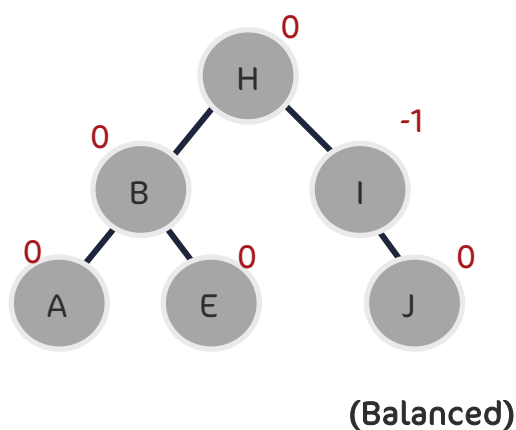
3. Insert E



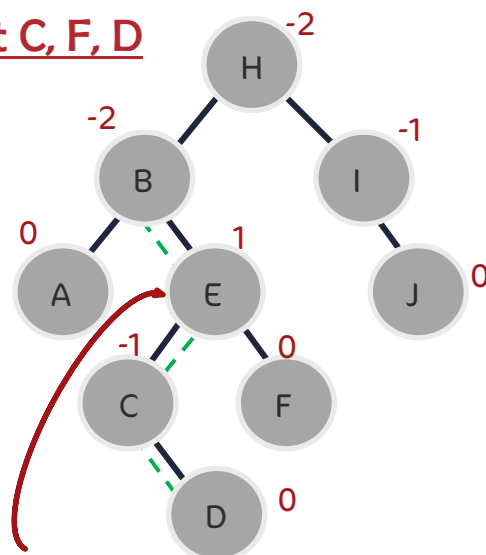
a) We first perform RR rotation on node B
The resultant balance tree after RR rotation is:



b) We first perform LL rotation on the node I
The resultant balance tree after LL rotation is:

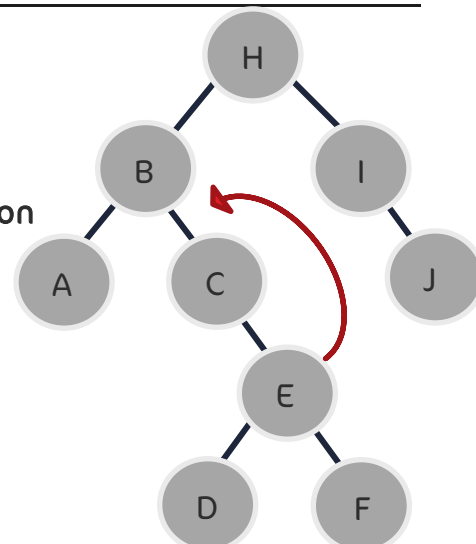


4. Insert C, F, D

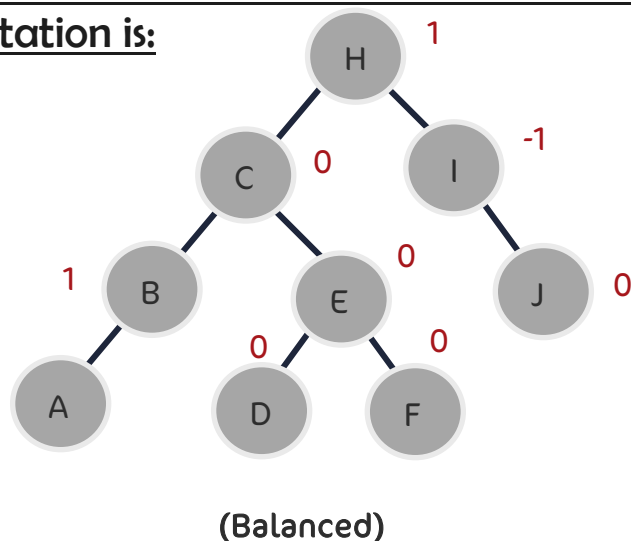


a) We first perform LL rotation on node E
The resultant balance tree after LL rotation is:

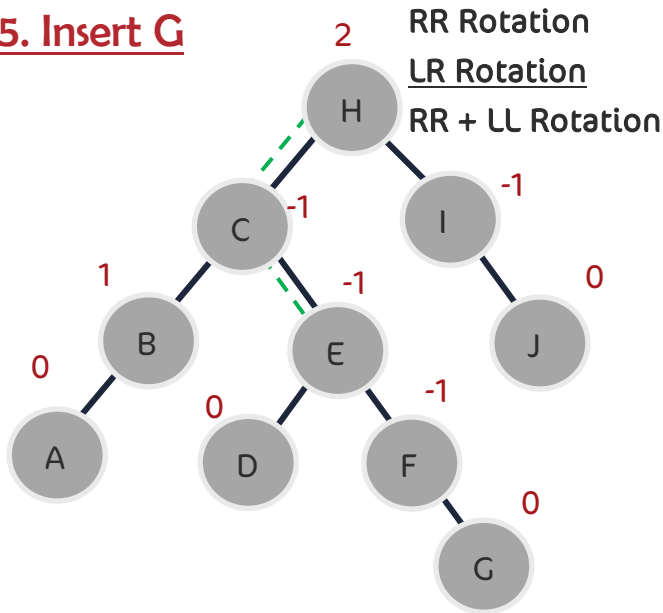
LL Rotation
 RL Rotation
 RR + LL Rotation



b) We then perform RR rotation on node B
The resultant balanced tree after RR rotation is:

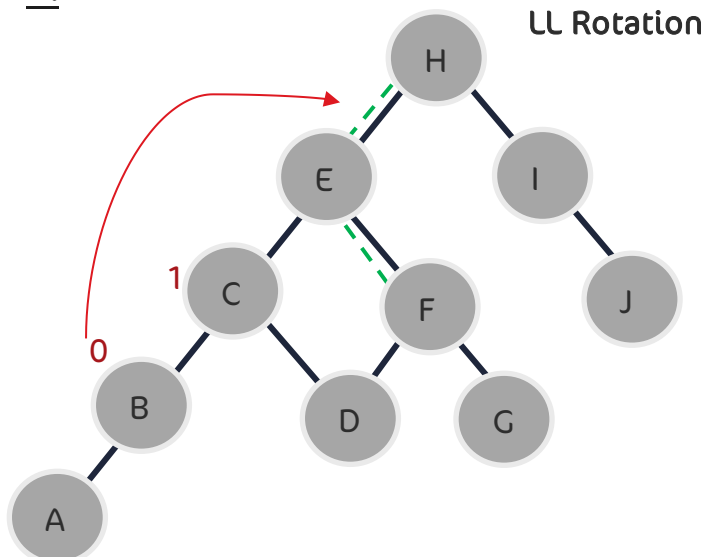


5. Insert G



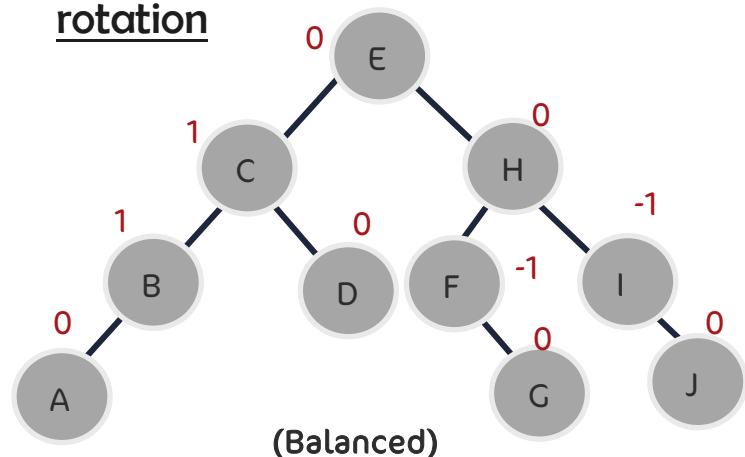
a) We first perform RR rotation on node C

The resultant tree after RR rotation is:

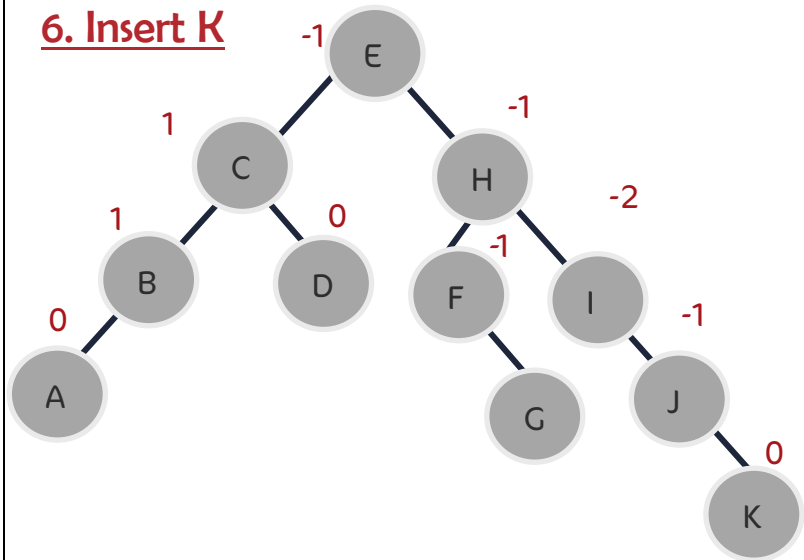


b) We then perform LL rotation on node H

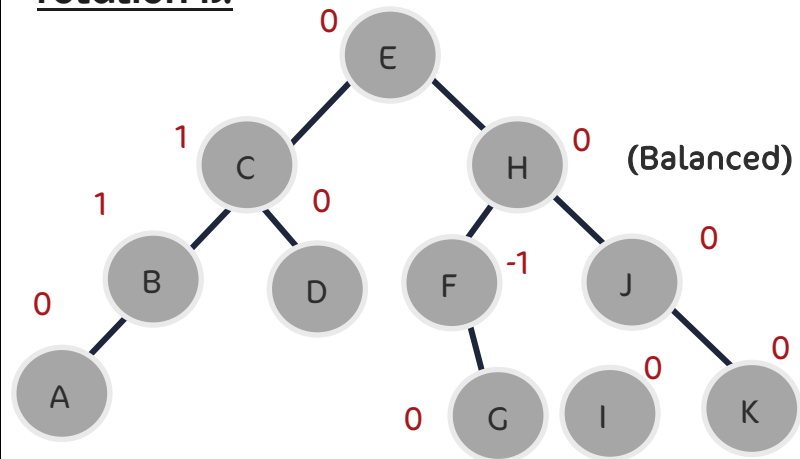
The resultant balanced tree after LL rotation



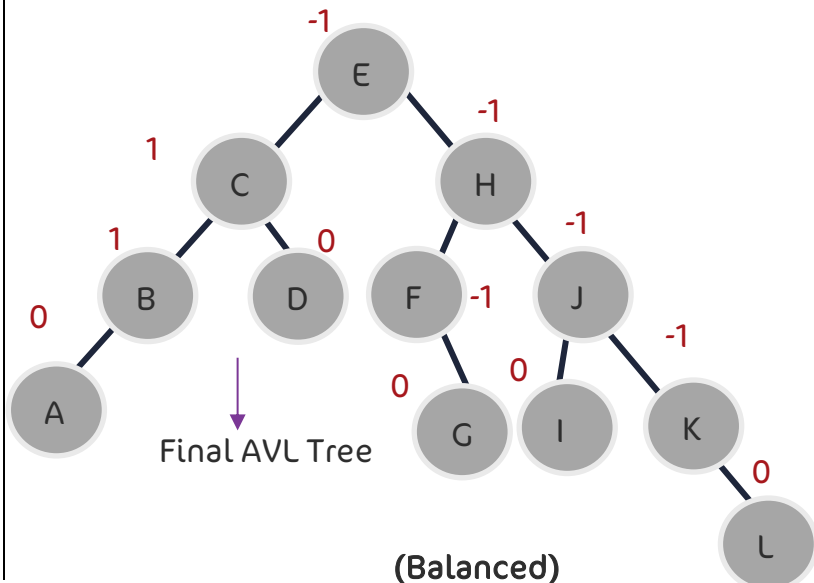
6. Insert K



The resultant balanced tree after RR rotation is:



7. Insert L



THE END